

Aquí tienes el documento estructurado y formateado, listo para que lo exportes a PDF y se lo compartas a tu equipo. Está diseñado para que cada encargado sepa exactamente qué tecnología usar, qué entidades crear y cómo interactuar con la arquitectura general usando Cursor/Windsurf.

Documento de Especificaciones

Técnicas: Backend

Proyecto: Plataforma "Last-Mile Delivery" (Logística de Última Milla)

1. Reglas Generales de Arquitectura (El Estándar del Equipo)

Para mantener el orden y cumplir con los requisitos del proyecto, todos los desarrolladores deben respetar las siguientes reglas:

- **Estructura Monorepo:** Todo el código vivirá en un solo repositorio. Cada microservicio tendrá su propia carpeta independiente (ej. `MS-FLOTA`, `MS-PEDIDOS`).
- **Contenedorización Aislada:** Dentro de la carpeta de cada microservicio, **debe existir un `Dockerfile`** que empaquete únicamente ese servicio con sus dependencias (`requirements.txt`, `pom.xml`, o `package.json`).
- **Un solo Archivo `.env` Global:** Está **estrictamente prohibido** tener archivos `.env` dentro de las carpetas de los microservicios. Todas las credenciales (bases de datos, puertos, usuarios) se guardarán en un único `.env` en la raíz del proyecto.
- **Orquestación Centralizada:** En la raíz del proyecto habrá un único archivo `docker-compose.yml`. Este archivo se encargará de leer el `.env` global, levantar las 3 bases de datos y construir los 5 microservicios simultáneamente, inyectando las variables de entorno correspondientes a cada uno.

2. Definición de los Microservicios Restantes (MS2 - MS5)

A continuación, se detalla la especificación para los microservicios que faltan desarrollar. Cada integrante puede usar esta guía como "Prompt" para su IA (Cursor/Windsurf).

Microservicio 2 (MS-PEDIDOS): Gestión de Envíos

- **Responsable:** [Nombre del integrante]
- **Stack Tecnológico:** Java (Spring Boot) + PostgreSQL (SQL 2).
- **Puerto Interno:** 8080
- **Objetivo:** Administrar la creación de paquetes, clientes de origen/destino y asignación básica.
- **Entidades y Relaciones:**

- Pedido (Tabla principal): id (PK), cliente_nombre (String), direccion_origen (String), direccion_destino (String), estado (String: 'creado', 'asignado', 'en_camino', 'entregado'), fecha_creacion (Date).
- Detalle_Paquete (Relación 1:N con Pedido): id (PK), pedido_id (FK), peso_kg (Float), dimensiones (String), contenido_desc (String).
- **Endpoints (API REST):**
 - POST /pedidos: Crea un pedido nuevo con sus detalles.
 - GET /pedidos: Lista todos los pedidos (con paginación).
 - PUT /pedidos/{id}/estado: Actualiza el estado del pedido.
- **Instrucción para IA:** "Desarrolla el MS-PEDIDOS en Spring Boot 3 con JPA/Hibernate. Usa las credenciales de PostgreSQL inyectadas por variables de entorno. Crea las entidades Pedido y Detalle_Paquete, los DTOs para request/response y un Controlador RESTful."

Microservicio 3 (MS-EVENTOS): Historial y Tracking

- **Responsable:** [Nombre del integrante]
- **Stack Tecnológico:** Node.js (Express o NestJS) + MongoDB (NoSQL).
- **Puerto Interno:** 3000
- **Objetivo:** Guardar el registro inmutable de todo lo que ocurre con un paquete (la bitácora de tiempo). Al ser NoSQL, es perfecto para guardar documentos JSON con mucha información variable.
- **Colección MongoDB (Eventos):**
 - Estructura del Documento: _id (ObjectId), pedido_id (Integer - Referencia a MS2), conductor_id (Integer - Referencia a MS1), tipo_evento (String: 'recogido', 'retraso', 'entregado'), descripcion (String), timestamp (Date), coordenadas (Object: {lat, lng}).
- **Endpoints (API REST):**
 - POST /eventos: Registra un nuevo evento en la bitácora.
 - GET /eventos/pedido/{pedido_id}: Devuelve la línea de tiempo completa de un pedido específico.
- **Instrucción para IA:** "Desarrolla el MS-EVENTOS en Node.js usando Express y Mongoose. Conéctate a MongoDB usando variables de entorno. Crea el esquema para la colección Eventos y los endpoints para insertar y consultar el historial por pedido_id."

Microservicio 4 (MS-ORQUESTADOR): Vista Unificada

- **Responsable:** [Nombre del integrante]
- **Stack Tecnológico:** Python (FastAPI).
- **Base de Datos:** NINGUNA (Regla de la rúbrica).
- **Puerto Interno:** 8001
- **Objetivo:** Actuar como intermediario (BFF - Backend For Frontend). El Frontend no llamará a MS1, MS2 y MS3 por separado; llamará a este orquestador, y este se encargará de hacer las peticiones HTTP internas (usando librerías como httpx o requests) para juntar la información.
- **Endpoints (API REST):**

- GET /dashboard/resumen: Hace un request a MS1 para traer total de conductores y a MS2 para traer total de pedidos, uniendo ambos JSON en una sola respuesta.
- GET /dashboard/envio/{pedido_id}: Consulta a MS2 los datos del paquete y a MS3 la línea de tiempo, devolviendo un JSON completo para la vista de detalle del Frontend.
- **Instrucción para IA:** *"Desarrolla el MS-ORQUESTADOR en FastAPI. NO uses bases de datos. Utiliza la librería httpx de forma asíncrona para consumir las APIs internas de MS-FLOTA (http://ms_flota_app:8000) y MS-PEDIDOS (http://ms_pedidos_app:8080) y consolida sus respuestas en nuevos schemas Pydantic."*

Microservicio 5 (MS-ANALÍTICA): Reportes AWS Athena

- **Responsable:** [Nombre del integrante] (Recomendado: El encargado de Data Science).
- **Stack Tecnológico:** Python (FastAPI + Boto3 para AWS).
- **Base de Datos:** AWS Athena (Consultando los CSV/JSON del Bucket S3).
- **Puerto Interno:** 8002
- **Objetivo:** Proveer endpoints analíticos al Frontend ejecutando queries SQL directamente sobre el catálogo de datos (AWS Glue/Athena) generado por el equipo de ingesta.
- **Endpoints (API REST):**
 - GET /analitica/entregas-por-mes: Ejecuta un query en Athena para contar envíos agrupados por mes.
 - GET /analitica/conductores-actividad: Ejecuta un query que cruza datos analíticos de conductores y eventos.
- **Instrucción para IA:** *"Desarrolla el MS-ANALITICA en FastAPI. Usa la librería boto3 para conectarte a AWS Athena. Crea endpoints que envíen consultas SQL a Athena, esperen la ejecución con get_query_execution, y retornen los resultados formateados en JSON extraídos de S3."*